Open Source Software

관리 대상 제외

About..

컴퓨터소프트웨어공학과

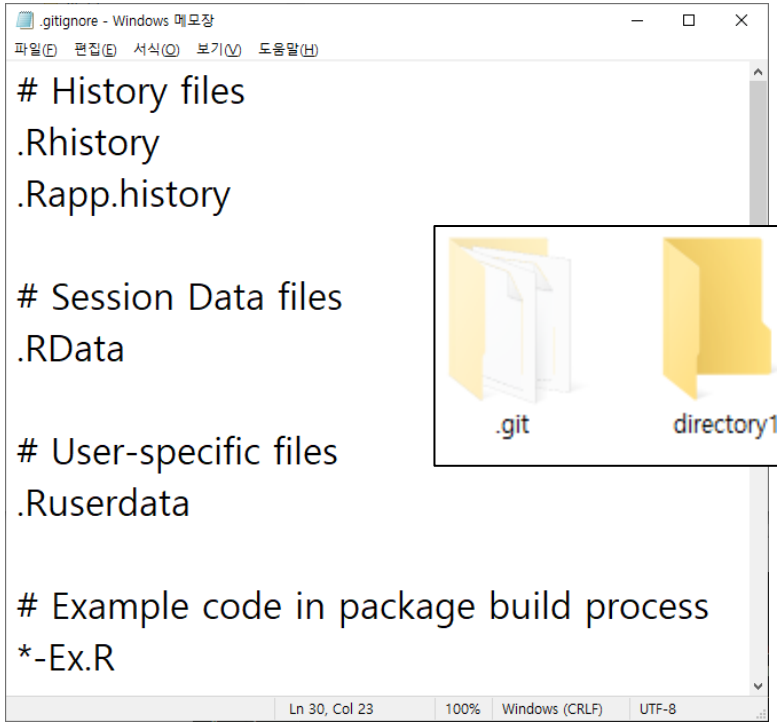
김 원 일

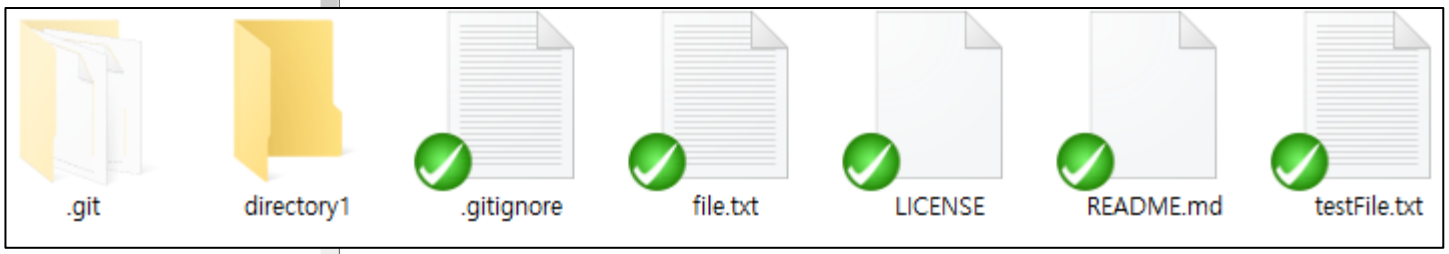
git의 관리 대상과 제외 대상

- **파일 이력 관리 대상**
  - git은 기본적으로 모든 종류의 파일을 대상으로 이력을 관리
  - 파일의 확장자 유무 등과 관계없이 무조건 관리 대상으로 지정
  - LFS(Large File Support)로 큰 파일까지도 관리 대상으로 지정 가능
  - 모든 파일을 관리 대상으로 지정하면 높은 비용이 발생
    - 빌드에서 발생하는 중간 단계의 파일은 관리할 필요 없음
    - 실행 파일도 소스 코드 변화에 따라 변화하므로 굳이 관리할 필요 없음
    - 실행 파일은 실행되는 운영체제와 시스템 환경에 따라 오동작할 수 있음
  - 관리 대상에서 제외할 파일이나 디렉터리를 기록할 파일 추가 가능
  - 기본적으로 “git init”을 통한 저장소 생성에는 만들어지지 않음
  - 별도 파일을 추가하여 관리 대상에서 제외시킬 수 있음

.gitignore 파일 – 1

- **파일 버전 관리 예외 정보 기록**
  - .(숨김 속성) + git + ignore(무시)
  - "."으로 시작하는 이유는 리눅스를 기반으로 하기 때문
  - 리눅스에서는 파일명이 "."으로 시작하면 숨김 파일
  - 사용자에게 노출할 필요가 없으므로 윈도우에서도 숨김 속성으로 처리





.gitignore 파일 – 2

- **파일과 디렉터리 모두 지정 가능**
  - #으로 시작되면 주석으로 처리되어 적용되지 않음
  - 상대, 절대 경로를 이용하여 특정 파일/디렉터리 지정도 가능
  - 예외로 지정되면 어떠한 추가/수정/삭제가 발생해도 관리하지 않음
  - .gitignore도 파일이기 때문에 수정하면 반드시 commit해야 함

.gitignore - Windows 메모장

파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)

```
#test
*.text

# History files
.Rhistory
.Rapp.history

# Session Data files
.RData

# User-specific files
.Ruserdata
```

Ln 32, Col 1

MINGW64:/d/__수업자료/2021/2학기/R/201007001

```
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   .gitignore

no changes added to commit (use "git add" and/or "git commit -a")

YUHAN@YP12624115 MINGW64 /d/___수 업 자 료 /2021/2학 기 /R/201007001 (main)
$
```

.gitignore 파일 – 3

- 파일과 디렉터리 모두 지정 가능
  - 확장자가 ".text"라면 파일에 대한 추적을 수행하지 않음
  - 새롭게 파일을 추가하더라도 추적 대상에 포함되지 않음
  - 디버깅 또는 임시 파일들을 추가하지 않도록 구성할 때 사용

MINGW64:/d/__수업자료/2021/2학기/R/201007001

```
YUHAN@YP12624115 MINGW64 /d/___수 업 자 료 /2021/2학 기 /R/201007001 (main)
$ ls
LICENSE  README.md  directory1/  file.txt  testFile.txt

YUHAN@YP12624115 MINGW64 /d/___수 업 자 료 /2021/2학 기 /R/201007001 (main)
$ vi file.text

YUHAN@YP12624115 MINGW64 /d/___수 업 자 료 /2021/2학 기 /R/201007001 (main)
$ ls
LICENSE  README.md  directory1/  file.text  file.txt  testFile.txt

YUHAN@YP12624115 MINGW64 /d/___수 업 자 료 /2021/2학 기 /R/201007001 (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

YUHAN@YP12624115 MINGW64 /d/___수 업 자 료 /2021/2학 기 /R/201007001 (main)
$
```

관리 대상 제외 실습 - 1

- ".gitignore" **파일**
  - **관리 대상 제외 파일/디렉터리 정보를 입력하는 파일**
  - **저장소의 최상위에 존재해야만 정상적으로 동작**
  - **관대 대상 제외 실습을 위한 저장소를 별도로 생성**
  - **저장소에** ".gitignore" **파일 생성 후 아래 내용 입력**
  - **입력 후 저장**(w)**하고 종료**(q)

MINGW64:/f/suho

```
# except file : *.text
*.text

# temp file
*.tmp

~
~
~
~
~
~
~
.gitignore[+] [unix] (08:59 01/01/1970)                    6,0-1 All
:wq
```

관리 대상 제외 실습 - 2

- **제외 적용 시점 확인**
  - 제외 대상 정보의 파일도 파일이기 때문에 저장소 추가가 필요
  - 아직 관리 대상이 아닌 상태에서 제외 대상 파일 생성
  - 정확한 적용 확인을 위해 적용 대상이 아닌 파일도 같이 생성

MINGW64:/f/suho

```
unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore

nothing added to commit but untracked files present (use "git add" to track)

unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$
```

관리 대상 제외 실습 - 3

- **제외 대상 파일 생성**
  - 관리 대상이 아닌 상태에서 파일이 존재하는 것만으로도 제외 적용
  - .gitignore 파일이 생성되면 그 즉시 적용되는 것을 확인 가능

MINGW64:/f/suho

```
unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ touch test.c

unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ touch test.text

unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        test.c

nothing added to commit but untracked files present (use "git add" to track)

unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$
```

관리 대상 제외 실습 - 4

- **제외 대상 파일 삭제**
  - 적용되어 있는 제외 대상 파일을 바로 삭제한 경우의 상태
  - 제외 대상 파일을 삭제하면 즉시 제외 대상 파일도 대상으로 변경
  - 즉, .gitignore 파일의 존재 유무 만으로 제외 여부를 즉시 판별

MINGW64:/f/suho

```
unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ rm -rf .gitignore

unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        test.c
        test.text

nothing added to commit but untracked files present (use "git add" to track)

unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$
```

관리 대상 제외 실습 - 5

- **제외 대상 파일 다시 추가**
  - 동일한 .gitignore 파일을 다시 생성한 다음의 상태
  - 제외 대상 파일인 test.text 파일이 대상에서 제외된 것을 확인 가능
  - 따라서 .gitignore 파일은 저장소 생성 시에 만드는 것이 좋음

MINGW64:/f/suho

```
unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ vi .gitignore

unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        test.c

nothing added to commit but untracked files present (use "git add" to track)

unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ |
```

관리 대상 제외 실습 - 6

- **상태 보관**
  - **추가 실습을 위해 현재 상태를 그대로 commit하여 보관**
  - **제외 파일은 그대로 저장소에 존재하는 상태**
    - **제외 파일은 저장소에서 지우거나 이동 또는 복사해도 git은 관여하지 않음**

MINGW64:/f/suho

```
unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ git add .gitignore test.c
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the ne
xt time Git touches it

unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ git commit -m "add test files"
[master (root-commit) 1b13bd2] add test files
 2 files changed, 6 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 test.c

unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ ls
test.c  test.text

unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ |
```

관리 대상 제외 파일 실습 - 1

- **파일 추가**
  - 저장소에 대상 제외와 관리 대상 파일을 각각 추가
  - 제외 대상 파일은 개수와 관계 없이 제외되는 것을 확인

MINGW64:/f/suho

```
unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ touch except.cpp

unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ touch allow.text

unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        except.cpp

nothing added to commit but untracked files present (use "git add" to track)

unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ ls
allow.text  except.cpp  test.c  test.text

unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$
```

관리 대상 제외 파일 실습 - 2

- **제외 대상 변경**
  - *.text **제외 문장 앞에** #**으로 해당 줄을 주석으로 처리**
  - **처리 후 저장하고 편집기 종료**

MINGW64:/f/suho

```
# except file : *.text
#*.text

# temp file
*.tmp

~
~
~
~
~
~
~
~
~
~
~
~
~
.gitignore[+] [unix] (21:40 21/03/2025)                    2,1 All
:wq
```

관리 대상 제외 파일 실습 - 3

- **파일 수정 후 확인**
  - .gitignore **파일 수정이 즉시 반영되는 것을 확인할 수 있음**
  - **현재 상태에서 모든 파일을 관리 상태로 변경**

MINGW64:/f/suho

```
unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ vi .gitignore

unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   .gitignore

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        allow.text
        except.cpp
        test.text

no changes added to commit (use "git add" and/or "git commit -a")

unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$
```

관리 대상 제외 파일 실습 - 4

- **관리 대상으로 모두 추가**
  - 생성한 모든 파일들을 관리 대상으로 추가
  - 제외 대상 파일들까지 모두 포함하여 관리대상으로 추가한 상태

MINGW64:/f/suho

```
unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ git add . ; git commit -m "add files"
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the ne
xt time Git touches it
[master c7b9823] add files
 4 files changed, 1 insertion(+), 1 deletion(-)
 create mode 100644 allow.text
 create mode 100644 except.cpp
 create mode 100644 test.text

unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ git status
On branch master
nothing to commit, working tree clean

unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ |
```

관리 대상 제외 파일 실습 - 5

- **파일 수정**
  - .gitignore **파일을 이전에 주석으로 처리했던 제외 대상의 주석 제거**
  - **이미** commit**한** allow.text **파일을 수정**
  - **제외 대상에 해당하는** add.text **파일을 생성하고 상태 확인**

MINGW64:/f/suho

```
# except file : *.text
*.text

# temp file
*.tmp

~
~
~
~
~
~
~
~
~
~
.gitignore[+] [unix] (22:54 21/03/2025)                    6,0-1 All
:wq
```

관리 대상 제외 파일 실습 - 6

- **수정 후의 상태**
  - 이전 확인에서 .gitignore 파일은 수정 후 즉시 적용되도록 구성
  - 새로 추가한 제외는 적용되나 이미 commit된 파일은 적용되지 않음

MINGW64:/f/suho

```
unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ vi .gitignore

unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ vi allow.text

unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ touch add.text

unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   .gitignore
        modified:   allow.text

no changes added to commit (use "git add" and/or "git commit -a")

unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$
```

관리 대상 제외 파일 실습 - 7

- .gitignore의 적용 시점 문제
  - 이미 commit된 파일에 대해서는 대상 제외가 적용되지 않음
  - 제외 대상 파일이라도 이미 commit되면 관리 대상에 포함
  - 즉, 대상 제외보다 commit된 파일에 대한 처리가 우선 적용
  - 따라서 .gitignore 파일은 저장소 생성 시점에 미리 적용하는 것이 좋음

  - 개발 과정 중에 어쩔 수 없거나 나중에 발견된 경우도 존재
  - 나중에 확인된 상태에서는 전체적으로 정리를 수행해야 함
  - 정리를 위해서는 git의 캐시를 삭제하는 방법이 가장 일반적
  - "git rm -r --cached ." 명령을 저장소 루트에서 실행

MINGW64:/f/suho

```
unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ git rm -r --cached .
rm '.gitignore'
rm 'allow.text'
rm 'except.cpp'
rm 'test.c'
rm 'test.text'
```

관리 대상 제외 파일 실습 - 8

- **캐시 삭제 후의 상태**
  - 캐시 삭제가 되면 저장소의 영향을 받는 모든 파일 삭제가 확인됨
  - 실제 삭제가 아니라 git 관리 대상에서의 삭제를 의미함
  - 변경이 없었던 관리 대상 파일도 "Untracked" 상태로 변경됨

MINGW64:/f/suho

```
unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        deleted:    .gitignore
        deleted:    allow.text
        deleted:    except.cpp
        deleted:    test.c
        deleted:    test.text

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        except.cpp
        test.c


unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ |
```

관리 대상 제외 파일 실습 - 9

- **파일 변화 적용**
  - "git add ."를 수행하고 상태를 확인하면 정리된 상태가 확인됨
  - 제외 대상 파일에서 commi된 파일들을 제거로 나타남
  - .gitignore 수정으로, 추가된 add.text는 목록 자체에서 제외됨

MINGW64:/f/suho

```
unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ git add .

unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   .gitignore
        deleted:    allow.text
        deleted:    test.text


unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ ls
add.text  allow.text  except.cpp  test.c  test.text

unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ |
```

관리 대상 제외 파일 실습 - 10

- commit으로 확정
  - 명령 실행하면, 이전에 commit된 2개의 파일이 삭제됨을 확인
  - 제외 대상은 수정해도 관리되지 않는 것을 확인할 수 있음

MINGW64:/f/suho

```
unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ git commit -m "modify .gitignore and apply"
[master 88c71be] modify .gitignore and apply
 3 files changed, 1 insertion(+), 1 deletion(-)
 delete mode 100644 allow.text
 delete mode 100644 test.text

unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ vi allow.text

unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ vi test.c

unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   test.c

no changes added to commit (use "git add" and/or "git commit -a")
```

관리 대상 제외 파일 실습 - 11

- .gitignore **다시 수정**
  - **제외 되었던 확장자를 다시 적용했을 때의 상태 확인**
  - **즉시 해당 확장자의 파일들이** "Untracked"**로 추가되는 것을 확인**

MINGW64:/f/suho

```
unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ vi .gitignore

unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   .gitignore
        modified:   test.c

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        add.text
        allow.text
        test.text

no changes added to commit (use "git add" and/or "git commit -a")

unangel@DESKTOP-I8OQG85 MINGW64 /f/suho (master)
$
```